

Framework de PHP Asíncrono y jQuery

Manual de referencia

Versión 2.1 (compatible con 2.0)

Revisión octubre 2026

Ricardo Seiffert
seifox@hotmail.es · github.com/seifox/APJ2

Contenido

Introducción

Novedades de la versión 2

Requerimientos

Instalación

- El archivo init.php
- El archivo APJPDO.ini.php
- Las carpetas de su aplicación
- Contenido de la carpeta APJ
- Namespaces y auto-carga

La vista

- Instrucción de precarga APJ:{ }
- APJCall()
- APJSubmit()
- APJSubmitSilent() y APJCallSilent()
- Los diálogos (jAlert)
- Personalizar el aspecto
- API JavaScript avanzada
- Versión multilenguaje

El controlador (APJController)

- Cómo se atiende una solicitud
- Normas generales
- Propiedades
- Métodos de la vista y el formulario
- Métodos comunes (también en el modelo)
- Responder a la vista
- Instrucciones jQuery: jQ()
- Diálogos desde el controlador
- Otras respuestas
- APJHtmlGen

El modelo (APJModel)

- Modelo en desarrollo
- Modelo en producción
- Propiedades
- Métodos de estructura y documentación
- Validación
- Otros métodos
- Métodos ORM
- SQL directo

Transacciones

Herramienta de modelos (apjtool.php)

Control de sesión (APJSession)

Iniciar sesión (login)

Métodos

Registro de errores (APJLog)

Buenas prácticas

Historial de versiones

Introducción

¿Qué es APJ?

APJ (Asynchronous PHP and jQuery) es un framework para PHP basado en el patrón MVC (Modelo-Vista-Controlador), con programación orientada a objetos, que utiliza jQuery para la manipulación del DOM. Está enfocado en el intercambio de datos de forma asíncrona por medio de AJAX, en formato JSON, entre el controlador y la vista. Su principal fortaleza es el desarrollo de aplicaciones y sistemas web.

Cuenta con un ORM, un manejador de datos simplificado basado en PDO (PHP Data Objects), que reduce el uso de sentencias SQL complejas para manipular la base de datos, sin impedir usarlas cuando hacen falta.

¿Por qué asíncrono?

Porque permite intercambiar información entre la vista (la página HTML) y el controlador (el programa PHP en el servidor) sin recargar la página, y retornar desde el controlador valores e instrucciones que modifican la página (el DOM) casi a su antojo.

Quien ha hecho aplicaciones web sabe el trabajo que significa repoblar un formulario que se envió para guardar, encontró errores de validación y debe quedar tal como estaba para que el usuario corrija. Con APJ el formulario nunca se recarga: el controlador responde con las instrucciones exactas (mostrar un mensaje, marcar un campo, refrescar una grilla) y la página se actualiza en su lugar.

¿Por qué jQuery?

Porque es la librería JavaScript más difundida, con una comunidad muy grande, que se mantiene actualizada y difícilmente desaparecerá. APJ 2 funciona con jQuery 3.7 y 4.0, y no depende de ningún otro framework de JavaScript ni de CSS.

¿Qué es MVC?

Modelo, Vista, Controlador es un patrón de arquitectura de software que separa los datos, la lógica y la interfaz de usuario, lo que permite la abstracción, la reutilización y la separación de responsabilidades.

Capa	Responsabilidad en APJ
Modelo	Todo lo que tiene que ver con datos: lectura, guardado, validaciones y lógica de negocio. Un modelo por tabla.
Vista	Lo que se ve en pantalla e interactúa con el usuario: la página HTML, con su estructura fija.
Controlador	Controla e interactúa entre la vista, la lógica del programa y los modelos. Recibe las solicitudes de la vista y le responde con instrucciones.

En APJ la vista y el modelo nunca se comunican directamente: todo pasa por el controlador. Por eso, más que un MVC clásico, se parece a un MVP (Modelo-Vista-Presentador).

¿Qué ventajas tiene?

- Código reducido, reutilizable y ordenado, que evita la duplicidad y facilita la mantención y la expansión de los proyectos.
- Cada capa se maneja por separado, lo que facilita entender un proyecto y trabajar en grupo.
- Evita el “código espagueti”: mezclar HTML, PHP, JavaScript y SQL en un mismo archivo.

Novedades de la versión 2

APJ 2 es una reescritura de APJ 1.x para PHP 8, con namespaces y tipos estrictos en el núcleo. Las ideas son las mismas, pero cambia la forma de escribir controladores, modelos y vistas.

Aspecto	APJ 1.x	APJ 2.x
PHP	5.4 o superior	8.1 o superior (2.1 probado en 8.3 y 8.5)
Namespaces	No (todo global)	`APJ\`, `Models\`, `Helpers\`, `Controllers\`
Carpeta del núcleo	`Libs/APJ/`	`APJ/`
Archivos del núcleo	`*.class.php`	`*.php`
Modelos	`Model_Nombre` en `Nombre.model.php`	`Models\Nombre` en `Models/Nombre.php`
Inicio de la vista	`var url = ...; var timeout = ...;`	`APJ.init(APJ: {APJInit()}).exposeGlobals();`
Respuesta JSON	Propiedades dinámicas	`jQAction` con `JsonSerializable`
Logs	`Libs/APJ/APJPDO_logs/`	`APJ/Logs/`
Vendor (Composer)	Opcional	Obligatorio (`vendor/autoload.php`)

Versión 2.1

La versión 2.1 prepara el framework para PHP 8.5 sin cambiar su API: un proyecto escrito para 2.0 funciona igual en 2.1. Se probó el núcleo completo con PHP 8.5.12 y 8.3.15, con todos los avisos activados (E_ALL), sin deprecaciones ni advertencias. Los cambios son:

- `APJModelTool::scan()`: ya no usa `null` como índice de array (deprecado en PHP 8.5).
- `APJController::setFormValues()`: un campo `datetime-local` vacío ya no llama a `strtotime(null)` y queda vacío (antes mostraba 1969-12-31).
- `APJPDOConnection`: `__wakeup()` reemplazado por `__unserialize()`, como recomienda PHP 8.5.
- `APJSession`: tolera que falten `HTTP_USER_AGENT`, `REMOTE_ADDR` o `SERVER_NAME` (procesos por consola, clientes sin User-Agent).
- `APJModel::_values()`: tolera estructuras escritas a mano con `Extra` nulo.

jqajaj.js 2.0.7

- `APJSubmitSilent()` y `APJCallSilent()`: igual que `APJSubmit()`/`APJCall()`, pero sin el indicador de "ocupado".
- Diálogos con estructura propia, creada una vez y reutilizada; el aspecto sale solo de `APJ/css/jqajaj.css`, que no depende del CSS del sitio.
- Con un diálogo abierto, el teclado queda confinado a él (Tab recorre sus botones) y al cerrarlo el foco vuelve donde estaba.
- En `jConfirm()`, el botón "No" solo cierra el diálogo; ya no ejecuta el callback.
- Un radio sin marcar ya no se envía con valor "0".

Requerimientos

- Servidor web Apache (o compatible) con **PHP 8.1 o superior**; recomendado PHP 8.3 o 8.5. Extensiones: PDO con `pdo_mysql`, `mbstring` e `iconv`.
- **MySQL o MariaDB** con el motor InnoDB (necesario para las transacciones). Con un DSN propio se pueden usar otras bases compatibles con PDO.
- **Composer**: el framework exige el archivo `vendor/autoload.php`, aunque no use librerías de terceros.
- Un navegador actualizado (Chrome, Edge, Firefox o Safari).
- Conocimientos básicos de PHP orientado a objetos, HTML, CSS, jQuery y SQL.
- Un editor de código o IDE, por ejemplo Visual Studio Code o NuSphere PhpED.

Instalación

La instalación es manual:

1. Descargue el framework desde https://github.com/seifox/APJ2 y copie el contenido en la carpeta de su proyecto.
2. Verifique que existan la carpeta `APJ`, el archivo `init.php` y las carpetas `Models`, `Views`, `Helpers` y `vendor` (esta última creada por Composer, con su `autoload.php`).
3. Edite `init.php` (ver la tabla siguiente).
4. Edite `APJ/APJPDO.ini.php` con los datos de conexión a la base de datos.
5. Abra el proyecto en el navegador: http://localhost/<carpeta>/

El archivo init.php

Cada controlador lo carga con `require_once("init.php");`. Define las constantes de la aplicación, registra el auto-cargador de clases y carga `vendor/autoload.php`.

Constante / variable	Uso
`DEVELOPMENT`	`true` en desarrollo (muestra todos los errores y usa `$domain`/`$rootUrl` de desarrollo); `false` en producción.
`APPNAME`	Nombre de la aplicación. También es el nombre (cifrado) de la sesión.
`ROOT`, `APP`	Carpeta raíz física del proyecto (por defecto `__DIR__`).
`APJ`, `APJLOGS`	Carpeta del núcleo (`APP/APJ`) y de los logs de error (`APJ/Logs`, se crea sola).
`CONTROLLERS`, `MODELS`, `VIEWS`, `HELPERS`, `VENDORS`, `IMAGES`	Carpetas de controladores (la raíz), modelos, vistas, helpers, librerías de Composer e imágenes.
`$domain` → `DOMAIN`	Dominio: `localhost` en desarrollo y el dominio real en producción.
`$rootUrl` → `ROOTURL`	Carpeta pública del proyecto, por ejemplo `/MiAplicacion` (o `/` si está en la raíz del dominio).
`LOGIN`	Controlador de inicio de sesión, al que redirige `sessionControl()`.
`LOGIN_ATTEMPTS`	Intentos de inicio de sesión permitidos (por defecto 3), para usar en su login.
`SESSION_LIMIT`	Segundos de inactividad antes de que caduque la sesión (por defecto 1800).
`PERSISTENT_CONNECTION`	Si la conexión PDO es persistente.
`EMULATE_PREPARES`	Si PDO emula las consultas preparadas (por defecto `true`).
`FORMATS`	Formatos de presentación usados por `format()` y `setFormat()` (ver abajo).
`TIMEZONE`	Zona horaria de PHP (por ejemplo `America/Santiago`).

La constante `FORMATS` es un arreglo serializado con estas claves:

Clave	Contenido
`int`	[decimales, separador decimal, separador de miles] para enteros. Por ejemplo `[0, ',', '.']`.
`decimal`	Lo mismo para decimales. Por ejemplo `[2, ',', '.']`.
`date`, `datetime`, `time`, `timestamp`	Formatos de fecha y hora de PHP. Por ejemplo `d-m-Y` y `d-m-Y H:i:s`.
`booleanTrue`, `booleanFalse`	Texto para mostrar un booleano (`tinyint`): por ejemplo “Si” y “No”.

`init.php` define además la función `isEntryPoint($file)`, que devuelve `true` si el archivo indicado es el script PHP que se está ejecutando. Sirve para que un controlador incluido desde otro no se auto-instancie:

```
if (isEntryPoint(__FILE__)) {
    new Consulta();
}
```

El archivo APJPDO.ini.php

Contiene los datos de conexión. Empieza con `<?php return NULL;?>` para que, si alguien lo pide por el navegador, no muestre su contenido.

```
<?php return NULL;?>
[DSN]
host = localhost
user = "usuario"
password = "contraseña"
dbname = "mi_base"
charset = utf8
```

- El usuario debe poder leer la estructura de las tablas (`SHOW FULL COLUMNS`) si va a usar `setTable()` en desarrollo.
- Si la contraseña tiene símbolos como `$`, enciérrela entre comillas.

Las carpetas de su aplicación

```
mi_aplicacion/
├── APJ/              Núcleo del framework
│   ├── css/          jqajaj.css (aspecto de los diálogos)
│   ├── images/       Íconos de los diálogos y del indicador de espera
│   ├── jQuery/       Copia local de jQuery
│   ├── js/           jqajaj.js y su versión multilenguaje
│   └── Logs/         Registro diario de errores (AAAA-MM-DD.txt)
├── Helpers/          Clases auxiliares (namespace Helpers\)
├── Models/           Modelos, uno por tabla (namespace Models\)
├── vendor/           Librerías de Composer (autoload.php)
├── Views/            Vistas HTML
├── css/, images/     Estilos e imágenes de las vistas
├── init.php          Constantes y arranque
├── apjtool.php       Herramienta de modelos (solo desarrollo)
└── controlador.php   Cada controlador va en la raíz del proyecto
```

Contenido de la carpeta APJ

Archivo	Descripción
`APJAutoload.php`	Auto-cargador de clases por namespace.
`APJCommon.php`	Trait con los métodos comunes de controladores y modelos (`format()`, fechas, etc.).
`APJController.php`	Controlador padre.
`APJHtmlGen.php`	Generador de HTML para usar en los controladores.
`APJLog.php`	Registro de errores en `APJ/Logs/`.
`APJModel.php`	Modelo padre (ORM).
`APJModelTool.php`	Herramienta para pasar modelos de desarrollo a producción (la usa `apjtool.php`).
`APJPDO.php`	Acceso a la base de datos sobre PDO; el modelo hereda de esta clase.
`APJPDO.ini.php`	Configuración de la conexión.
`APJPDOConnection.php`	Conexión PDO única (singleton) compartida por todos los modelos.
`APJSession.php`	Control de sesión de usuarios.
`jQ.php`, `jQSelector.php`, `jQAction.php`	Arman la respuesta JSON con las instrucciones jQuery y las acciones para la vista.
`js/jqajaj.js`, `jqajaj.min.js`	Módulo JavaScript de comunicación con el controlador y de diálogos.
`js/jqajaj_multi.js`, `jqajaj_multi.min.js`	La misma versión, con textos en varios idiomas.
`css/jqajaj.css`, `jqajaj.min.css`	Aspecto de los diálogos y del indicador de espera.

Namespaces y auto-carga

El auto-cargador convierte el namespace de una clase en una ruta de archivo:

Namespace	Carpeta	Ejemplo
`APJ\`	`APJ/`	`APJ\APJModel` → `APJ/APJModel.php`
`Models\`	`Models/`	`Models\Contactos` → `Models/Contactos.php`
`Helpers\`	`Helpers/`	`Helpers\Sendmail` → `Helpers/Sendmail.php`
`Controllers\`	raíz del proyecto	`Controllers\Consulta` → `consulta.php` (o `Consulta.php`)

El nombre del archivo debe coincidir con el de la clase, respetando mayúsculas y minúsculas: en Windows no se nota, pero en un servidor Linux sí. Los controladores son la excepción: si no encuentra `Consulta.php`, el auto-cargador prueba con el nombre en minúsculas (`consulta.php`), porque así suelen nombrarse por la URL.

La vista

Las vistas son páginas HTML que se guardan en la carpeta `Views`. Contienen la estructura fija de la página; el controlador solo regenera las partes que cambian (una grilla, un formulario, un mensaje).

Ejemplo de la sección `<head>` de una vista:

```
<head>
  <meta charset="UTF-8">
  <title>APJ:{tituloVista()}</title>
  <script src="APJ/jQuery/jquery-4.0.0.min.js"></script>
  <script src="APJ/js/jqajaj.min.js?ver=2.0.7"></script>
  <script>
    APJ.init(APJ:{APJInit()}).exposeGlobals();
  </script>
  <link rel="stylesheet" href="css/micss.css">
  <link rel="stylesheet" href="APJ/css/jqajaj.css?ver=2.0.7">
</head>
```

Línea	Para qué sirve
`jquery-4.0.0.min.js`	Carga jQuery (versión 3.7 o 4.0) desde la copia local.
`jqajaj.min.js`	Carga el módulo de comunicación con el controlador y de diálogos.
`APJ.init(APJ:{APJInit()})`	El controlador reemplaza `APJ:{APJInit()}` por `{"url":"controlador.php","timeout":10000}`: la dirección del controlador con el que habla la vista y el tiempo máximo de espera.
`.exposeGlobals()`	Deja disponibles como funciones globales `APJCall`, `APJSubmit`, `jInfo`, `jConfirm`, etc., para usarlas directamente en atributos `onclick`.
`jqajaj.css`	Aspecto de los diálogos. Se puede modificar a gusto (ver “Personalizar el aspecto”).

> **Nota:** El parámetro `?ver=2.0.7` evita que el navegador use una copia vieja guardada en caché. Cámbielo cada vez que actualice `jqajaj.js` o `jqajaj.css`; si sube uno sin el otro, los diálogos pueden verse rotos.

El controlador inserta automáticamente `<base href="ROOTURL/">` al inicio del `<head>`, de modo que las rutas relativas de la vista (`APJ/...`, `css/...`) funcionan aunque el sitio esté en una subcarpeta.

Instrucción de precarga APJ:{ }

Permite ejecutar un método del controlador al cargar la vista y reemplazar la instrucción por lo que ese método retorne. Se puede poner en cualquier parte de la vista.

```
<tbody id="grillaCiudades">
  APJ:{grillaCiudades(CL)}
</tbody>
```

Al mostrar la vista, el controlador ejecuta `grillaCiudades('CL')` y pone en su lugar lo que retorne. Sirve para precargar una grilla, las opciones de un `<select>`, el título de la página, etc.

- Los argumentos se separan por comas y llegan como texto. Las comillas que los rodeen se quitan: `APJ:{m('PESOS')}` y `APJ:{m(PESOS)}` son equivalentes.

- Sin argumentos (`APJ:{nada()}`) el método se llama sin parámetros, así que se aplican sus valores por defecto.
- Si el método no existe o retorna `null`, la instrucción se reemplaza por un texto vacío.

APJCall()

Llama a un método público del controlador, con argumentos opcionales:

```
APJCall('metodo', argumento1, argumento2, ...);
```

```
<!-- 1. Desde un evento -->
<button type="button" onclick="APJCall('mostrarDatos', 10, 'Ascendente');">MOSTRAR</button>

<!-- 2. Con jQuery -->
<button type="button" id="mostrar">MOSTRAR</button>
<script>
  $('#mostrar').on('click', function () { APJCall('mostrarDatos', 10, 'Ascendente'); });
</script>
```

En el controlador, los argumentos llegan por separado: `public function mostrarDatos($cantidad, $orden)`. Si el controlador define `$useParametersAsArray = true`, llegan juntos en un arreglo.

> **Importante:** `APJCall()` **no envía el formulario**: solo llegan los argumentos indicados. Si el método necesita datos del formulario, páselos como argumentos o use `APJSubmit()`.

> **Importante:** Los botones que llaman a `APJCall()` o `APJSubmit()` deben ser `type="button"`; si no, el formulario además se envía de la forma tradicional y la llamada se duplica.

APJSubmit()

Envía un formulario al método indicado del controlador, con argumentos opcionales:

```
APJSubmit('idFormulario', 'metodo', argumento1, argumento2, ...);
```

```
<form id="formul">
  <!-- Envía el formulario a cambioSeleccion() con el valor elegido como argumento -->
  <select name="estado" onchange="APJSubmit('formul', 'cambioSeleccion', this.value);">
    <option value="1">Activo</option>
    <option value="0">Inactivo</option>
  </select>
  <!-- Envía el formulario a guardar() -->
  <button type="button" onclick="APJSubmit('formul', 'guardar');">GUARDAR</button>
</form>
```

En el controlador, los campos del formulario quedan en el objeto `$this->Form` (por ejemplo `$this->Form->estado`) y los argumentos adicionales en el arreglo `$this->Form->parameters`.

Un checkbox marcado envía su `value` (o “1” si no tiene) y uno sin marcar envía “0”. De un grupo de radios solo se envía el marcado.

APJSubmitSilent() y APJCallSilent()

Tienen exactamente la misma firma que `APJSubmit()` y `APJCall()`, pero no muestran el indicador de "ocupado" ni bloquean el teclado mientras esperan la respuesta.

Úselas en llamadas disparadas por `onchange` u `onblur` que solo calculan o buscan datos (completar un nombre a partir de un RUT, recalcular un total), para no interrumpir al usuario mientras pasa de un campo a otro con Tab. Deje `APJSubmit()`/`APJCall()` normales para las acciones que guardan datos o que el usuario dispara con un clic.

```
<input id="rut" name="rut" onblur="APJCallSilent('buscarPorRut', this.value);">
```

Los diálogos (jAlert)

`jInfo()`, `jWarning()`, `jError()`, `jConfirm()`, `jPrompt()` y `jProcess()` reemplazan a `alert()`, `confirm()` y `prompt()` de JavaScript. Ponen un fondo semitransparente sobre la página, así que el usuario debe responder antes de seguir. Mientras un diálogo está abierto, el teclado queda confinado a él: Tab recorre sus botones y al cerrarlo el foco vuelve al elemento donde estaba.

Se pueden llamar desde la vista (JavaScript) o desde el controlador (`$this->jInfo(...)`, con los mismos argumentos). El mensaje acepta HTML, por ejemplo `<br>` o `<b>`.

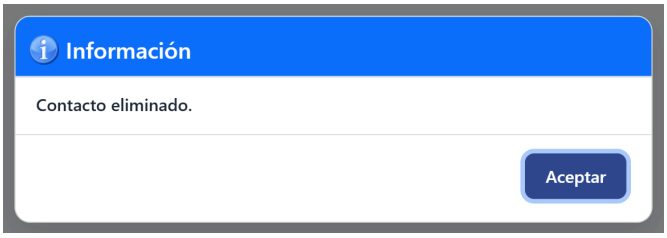
El callback se ejecuta al presionar Aceptar (o "Sí") y recibe los argumentos del arreglo indicado. Puede ser una función o el nombre de una función global como texto, por ejemplo `APJCall`.

jInfo()

```
jInfo('mensaje', 'título', callback, [arg1, arg2, ...]);
```

Muestra información; el usuario confirma con Aceptar. Solo el mensaje es obligatorio; el título por defecto es "Información".

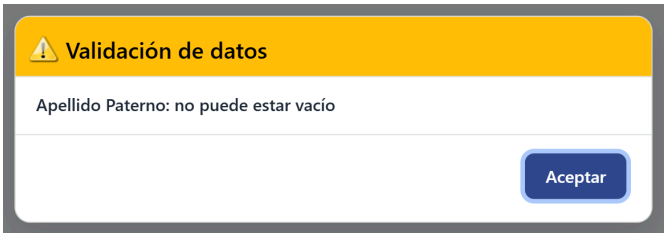
```
jInfo('Contacto eliminado.');
jInfo('Contacto eliminado.', 'Información', proceso, [1, 'Si']);  // al aceptar: proceso(1,
'Si')
```



jWarning()

Igual que `jInfo()`, para advertencias; es el que corresponde a los errores de validación.

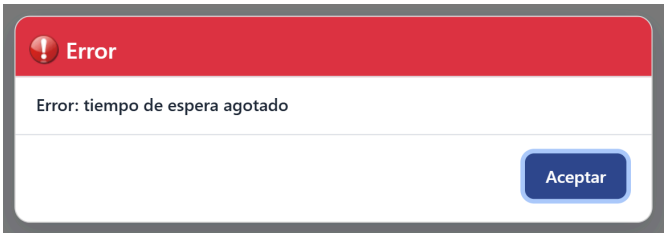
```
jWarning('Apellido Paterno: no puede estar vacío', 'Validación de datos');
```



jError()

Igual que los anteriores, para errores reales (de base de datos, de comunicación, etc.). El título por defecto es "Error".

```
jError('Error: tiempo de espera agotado');
```

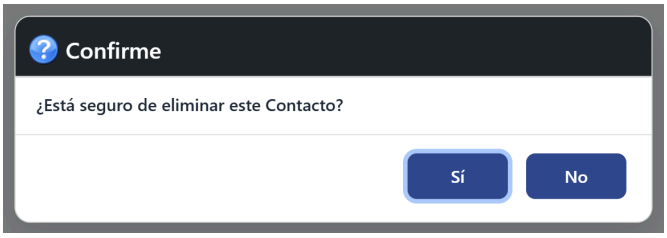


jConfirm()

Pide confirmar una acción. Solo si se responde "Sí" se ejecuta el callback, que por eso es obligatorio; "No" solo cierra el diálogo.

```
jConfirm('¿Está seguro de eliminar este Contacto?', 'Confirme', APJCall, ['eliminar', 101]);
```

Aquí el callback es la propia función `APJCall`: al presionar "Sí" se ejecuta `APJCall('eliminar', 101)`.

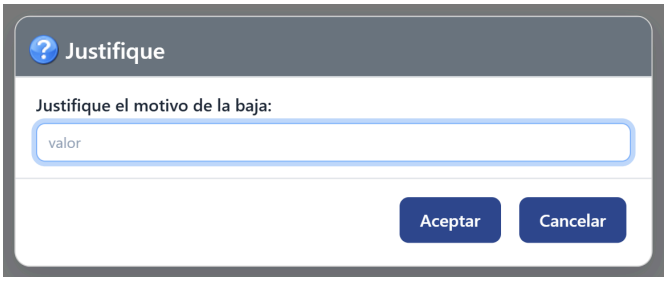


jPrompt()

Pide un valor. Lo ingresado se pasa al callback como **último** argumento, después de los argumentos del arreglo.

```
jPrompt('Justifique el motivo de la baja', 'Justifique', APJCall, ['darDeBaja']);
// al aceptar: APJCall('darDeBaja', valorIngresado)
```

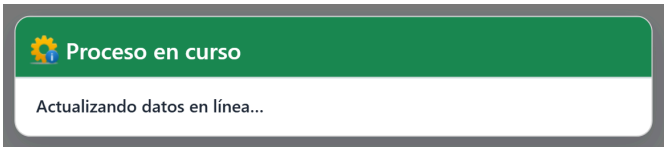
> **Nota:** Si el usuario presiona Cancelar, el callback se llama con un único argumento `null`. Si necesita distinguir la cancelación, use una función propia como callback y revise el valor antes de llamar al controlador.



jProcess()

```
jProcess('mensaje', 'título', 'estilo');
```

Indica que hay un proceso en curso. No tiene botones: se cierra con `jClose()` o al abrir cualquier otro diálogo. El estilo opcional `'blink'` hace parpadear el mensaje.



jClose() y jBusy()

`jClose()` cierra cualquier diálogo abierto o el indicador de espera. `jBusy()` muestra el indicador de espera (el mismo que aparece durante cada `APJCall()`/`APJSubmit()`), que bloquea el teclado hasta que se cierra.

Personalizar el aspecto

El aspecto de los diálogos y del indicador de espera sale solo de `APJ/css/jqajaj.css` y de los íconos de `APJ/images/` (`info32.png`, `warning32.png`, `error32.png`, `question32.png`, `process32.png` y `busy32.gif`). Para cambiar colores, tamaños o íconos, edite esos archivos.

`jqajaj.css` es autosuficiente: no depende de ningún otro CSS ni framework, y las reglas generales del sitio (por ejemplo `span { color: #000 }` o `button { ... }`) no lo afectan. Para lograrlo tiene un bloque "Aislamiento" que no hace falta modificar, y bloques de "Aspecto" que son los que se adaptan. La estructura de un diálogo es:

```
#dialogoverlay                          fondo semitransparente
#dialogbox.<tipo>Dialog                 cuadro (tipo: info, warning, error,
  #dialogboxhead                                confirm, prompt, process)
    #dialogboxicon + #dialogboxtitle    ícono y título
  #dialogboxbody > #dialogboxtext       mensaje (y #prompt_value en jPrompt)
  #dialogboxfoot                        botones #dialogOk y #dialogCancel
#busyoverlay + #busybox                 indicador de espera
```

Por ejemplo, para cambiar el color del encabezado de las advertencias:

```
.warningDialog #dialogboxhead { background: #ffc107; color: #000; }
```

> **Nota:** Una regla del sitio con `!important` sí puede pisar el aspecto del diálogo; evite usarlo en reglas generales. El HTML que usted ponga dentro del mensaje (`<b>`, `<table>`, etc.) sí recibe el CSS del sitio.

API JavaScript avanzada

Todo jqajaj vive en el objeto global `APJ`. `exposeGlobals()` solo crea atajos globales para lo más usado; todo está también disponible como `APJ.APJCall()`, `APJ.dialog.info()`, etc.

APJ.init(opciones)

Opción	Descripción
`url`	Controlador con el que se comunica la vista. Obligatoria (la entrega `APJInit()`).
`timeout`	Tiempo máximo de espera en milisegundos (por defecto 10000). Con `0` espera sin límite.
`images`	Carpeta de los íconos de los diálogos (por defecto `APJ/images/`).
`keyLock`	Si el indicador de espera bloquea el teclado y un diálogo lo confina (por defecto `true`).

Consulta periódica: APJ.poll()

```
APJ.poll('metodo', intervaloMs, parametros, { maxErrors: 0, onError: 'silent', immediate: false
});
APJ.stopPoll('metodo');
APJ.stopAllPolls();
```

Llama a un método del controlador cada cierto intervalo, sin indicador de espera; sirve, por ejemplo, para revisar mensajes nuevos o mantener viva la sesión. `onError` puede ser `'silent'` (seguir), `'stop'` (detenerse) o una función. Si el servidor responde algo que no es JSON (sesión caducada), el poll se detiene y se llama a `APJ.onSessionEnd`.

```
APJ.poll('mantenerSesion', 240000);              // cada 4 minutos
APJ.poll('revisarMensajes', 5000, [idSala], { immediate: true });
```

Datos desde el controlador: APJ.data

El controlador puede enviar datos a la vista con `jQ::setData($clave, $valor, $callback)`. En la vista se reciben definiendo una función en `APJ.data` con el nombre del callback (por defecto `defaultCallBack`):

```
// En la vista
APJ.data.recibeTotales = function (clave, valor) {
  console.log(clave, valor.neto, valor.iva);
};
```

```
// En el controlador
\APJ\jQ::setData('totales', ['neto' => 1000, 'iva' => 190], 'recibeTotales');
```

Eventos

Propiedad	Se llama cuando
`APJ.onComplete`	Termina cualquier `APJCall()`/`APJSubmit()`, con o sin éxito.
`APJ.onSessionEnd`	El servidor responde algo que no es JSON; normalmente porque la sesión caducó y el servidor devolvió la página de login.

Otros

- `APJ.dialog`: el módulo de diálogos (`APJ.dialog.info()`, `.warning()`, `.error()`, `.confirm()`, `.prompt()`, `.process()`, `.busy()`, `.close()`).
- `APJ.call(parametros)`: envío AJAX genérico al controlador, con los parámetros tal cual.
- `APJ.jShowDown(idInput, idContenedor)`: muestra u oculta el contenedor justo bajo el campo (ver `jShowDown()` en el controlador).
- Errores de comunicación: un controlador inexistente (404), un error del servidor (500) o un tiempo de espera agotado se muestran automáticamente con `jError()`.

Versión multilenguaje

`jqajaj_multi.js` (y `.min.js`) es la misma versión de jqajaj con los textos de los diálogos en español, inglés, portugués, francés y alemán. Se usa en lugar de `jqajaj.js` e indicando el idioma en `APJ.init()`:

```
APJ.init({ url: 'controlador.php', lang: 'en' }).exposeGlobals();
```

Con `translations` se reemplazan solo algunos textos, y con `APJ.addLang()` se agrega un idioma completo:

```
APJ.init({
  url: 'controlador.php',
  lang: 'es',
  translations: { buttons: { yes: 'Confirmar', no: 'Volver' } }
});

APJ.addLang('it', { titles: {...}, buttons: {...}, prompt: {...}, http: {...} });
```

El controlador (APJController)

El controlador es el corazón de cada módulo: muestra la vista, recibe sus solicitudes, usa los modelos y responde con instrucciones para la página. Cada controlador hereda de `APJ\APJController` y vive en la raíz del proyecto; el mismo archivo es la clase y el punto de entrada.

```
<?php
namespace Controllers;
require_once("init.php");

use APJ\APJController;
use APJ\APJHtmlGen;
use Models\Contactos;

class Consulta extends APJController
{
    private ?Contactos $modeloContactos = null;
    private ?APJHtmlGen $gen = null;

    public function __construct()
    {
        $this->TimeOut = 20000;
        $this->sessionControl();
        $this->modeloContactos = new Contactos();
        $this->gen = new APJHtmlGen();
        parent::__construct('consulta.html');
    }

    public function guardar(): void
    {
        // llamado desde la vista con APJSubmit('formContacto', 'guardar')
    }
}

new Consulta();
```

Línea	Explicación
`namespace Controllers;`	Los controladores van en el namespace `Controllers`.
`require_once("init.php");`	Carga las constantes, el auto-cargador y `vendor/autoload.php`.
`use ...`	Importa las clases del framework y los modelos que usa el controlador.
`private ?Contactos $modeloContactos`	Una propiedad por cada modelo u objeto auxiliar. Deben **declararse**: desde PHP 8.2, asignar una propiedad no declarada genera un aviso que puede corromper la respuesta.
`$this->TimeOut = 20000;`	Tiempo máximo de espera de la vista, en milisegundos (por defecto 10000). Con `0` espera sin límite, útil para procesos largos. Se asigna **antes** de `parent::__construct()`.
`$this->sessionControl();`	Exige sesión iniciada y acceso a este controlador; si no, redirige a `LOGIN`.
`parent::__construct('consulta.html')`	Atiende la solicitud: si viene de `APJCall()`/`APJSubmit()` ejecuta el método pedido y responde; si no, muestra la vista.
`new Consulta();`	Instancia el controlador al ejecutar el archivo.

Cómo se atiende una solicitud

- **Carga de página** (sin `action`): se muestra la vista con `render()`, que resuelve las instrucciones `APJ:{...}`.
- **Solicitud asíncrona** (`APJCall()`/`APJSubmit()`): el constructor detecta el método pedido, llena `$this->Form` con el formulario, ejecuta el método y responde con el JSON de instrucciones. La vista nunca se vuelve a cargar.
- Si el método no existe, la vista muestra "El método X no existe!".
- Si el método lanza una excepción no capturada, se registra el detalle en el log (`APJ/Logs/`) y la vista muestra "Error interno del servidor.", sin exponer información interna.
- Cualquier salida suelta (un `echo`, un aviso de PHP) se descarta antes de responder, para no corromper el JSON. Aun así, corrija los avisos en su origen.

Normas generales

- Un controlador por archivo, en la raíz del proyecto, con `namespace Controllers;` y heredando de `APJController`.
- Los métodos que se llaman desde la vista deben ser **públicos**.
- Toda la lógica va en el controlador (y la de negocio en el modelo), no en JavaScript de la vista: el controlador responde con instrucciones `$this->jQ(...)`.
- El HTML dinámico se arma con `APJHtmlGen`; la vista tiene la estructura fija y el controlador solo regenera el fragmento que cambia.

Propiedades

Propiedad	Descripción
`TimeOut` (int)	Tiempo máximo de espera de la vista, en milisegundos. Por defecto 10000; `0` = sin límite.
`userId`	Id del usuario con sesión iniciada; lo asigna `sessionControl()`.
`canRender` (bool)	Si se puede mostrar la vista.
`Form` (objeto)	Campos del formulario enviado con `APJSubmit()`. Los argumentos adicionales quedan en `$this->Form->parameters` (arreglo).
`useParametersAsArray` (bool)	Si los argumentos de `APJCall()` llegan como un arreglo (`true`) o como parámetros separados (`false`, por defecto).
`fieldTypes` (array)	Tipos especiales de campos para `setFormValues()`; se asignan con `setFieldType()`.
`lastPage`, `currentPage`, `previousPage`, `nextPage`	Resultado de la última paginación hecha con `paging()`.

Métodos de la vista y el formulario

`render(string $page, bool $return = false)`

Muestra la vista `$page` (de la carpeta `Views`) resolviendo las instrucciones `APJ:{...}`. Con `$return = true` la retorna como texto en vez de mostrarla. Lo llama `parent::__construct()`, así que rara vez se usa directamente.

`sessionControl(bool $controllerControl = true)`

Exige una sesión activa; si no la hay, redirige a `LOGIN`. Con `true` además exige que el usuario tenga acceso a este controlador (ver el capítulo de sesión). Con `false` solo exige sesión; úselo en controladores comunes a todos los usuarios. Se llama al inicio del constructor.

`formToModel($model)`

Pasa al modelo los campos del formulario enviado cuyos nombres coinciden con columnas del modelo.

`formObjectToModel($model)`

Pasa al modelo los valores de `$this->Form` que coinciden con sus columnas. Permite modificar `$this->Form` antes.

```
public function guardar(): void
{
    $this->Form->fecha_actualizacion = $this->currentDateTime();
    $this->formObjectToModel($this->modeloContactos);
    if ($this->modeloContactos->guardar() === false) {
        $this->jError($this->modeloContactos->errormsg);
        return;
    }
    $this->jInfo('Los datos se han guardado');
}
```

`modelToForm($model)`

Copia los valores del modelo (la fila leída con `find()`) a `$this->Form`. Es el inverso de `formObjectToModel()`.

`arrayToForm(array $array)`

Copia un arreglo asociativo a `$this->Form`.

`clearForm() / createForm()`

`clearForm()` elimina el objeto `Form`; `createForm()` lo crea vacío si no existe.

`setFormValues($data = '', ?string $form = null)`

Llena el formulario de la vista con un arreglo u objeto (o con `$this->Form` si no se indica `$data`). Cada clave se busca por **id** del campo; si se indica `$form`, se busca por **name** dentro de ese formulario. Usa `fieldTypes` para los checkbox, radio y `datetime-local`.

```
public function editaContacto(int $id): void
{
    if ($this->modeloContactos->find($id)) {
        $this->modelToForm($this->modeloContactos);
        $this->setFieldType('activo', 'checkbox');
        $this->setFormValues();
        $this->jQ('#nombres')->focus();
    }
}
```

`setFieldType(string $field, string $type = 'checkbox')`

Indica el tipo especial de un campo para `setFormValues()`: `checkbox`, `radio`, `datetime-local` o `file` (este último se omite). Los checkbox y radio deben tener su atributo `value`.

`getForm()`

Llena `$this->Form` con los datos enviados. Se ejecuta solo en cada solicitud asíncrona.

`options(array $array, string $valueIndex, string $textIndex, string|array|null $selected = '')`

Genera los `<option>` de un `<select>` a partir de un arreglo de filas. `$selected` puede ser un valor o un arreglo de valores (selección múltiple).

```
// En la vista: <select id="pais">APJ:{optPaises()}</select>
public function optPaises(): string
{
    $paises = $this->modeloPaises->all('pais');
    return $this->options($paises, 'codigo', 'pais', 'CL');
}
```

`paging(array $data, int $limit = 20, int $page = 1)`

Retorna una página de un arreglo y actualiza `lastPage`, `currentPage`, `previousPage` y `nextPage`.

`APJInit(): string`

Retorna el JSON de configuración para `APJ.init()` en la vista (`url` y `timeout`). Se usa como `APJ: {APJInit()}`.

`self() / getController() / classBasename()`

`self()` retorna el archivo del controlador (por ejemplo `consulta.php`); `getController()`, el nombre sin extensión (`consulta`); `classBasename()`, el archivo donde está definida la clase.

`timeout(): int`

Retorna `TimeOut`.

`executeLocally(string $file): bool (estático)`

Igual que `isEntryPoint()`: indica si el archivo es el que se está ejecutando.

`getLocalContent(string $url) / getContent(string $url)`

Leen un archivo local o remoto y retornan su contenido, o `null` si no se puede.

`absoluteUrl(string $path = ''): string`

Arma una dirección absoluta para el ambiente actual (protocolo + `DOMAIN` + `ROOTURL` + ruta), detectando HTTPS aun detrás de un proxy. Úselo para links en correos, códigos QR, etc.

```
$link = $this->absoluteUrl('/verifica.php?t=' . $token);
// https://midominio.cl/miapp/verifica.php?t=...
```

`redirect(string $url, bool $parent = false)`

Redirige a otra página. En una solicitud asíncrona lo hace la vista; con `$parent = true` redirige la ventana contenedora (útil dentro de un `<iframe>`). Termina la ejecución.

Métodos comunes (también en el modelo)

`format($value, string $type)`

Formatea un valor según `FORMATS` de `init.php`. Tipos: decimales (`float`, `real`, `double`, `decimal`), enteros (`smallint`, `mediumint`, `int`, `integer`, `bigint`, `bit`), booleanos (`tinyint`, `bool`, `boolean`) y fechas (`date`, `datetime`, `time`, `timestamp`). Una fecha vacía o nula retorna `null`.

```
$this->format(1234.5, 'decimal');      // "1.234,50"
$this->format('2026-10-22', 'date');   // "22-10-2026"
```

`convertDateTime(string $dateTime, string $format = 'Y-m-d H:i:s')`

Convierte una fecha escrita en distintos formatos, incluso con el mes en palabras (“22 de octubre de 2026”, “Oct 22, 2026”, “22/10/2026”), al formato indicado.

`verifyDate($date, string $format, bool $strict = true): bool`

Retorna `true` si la fecha es válida en el formato dado (o un timestamp Unix con `'timestamp'`).

`currentDateTime(string $format = 'Y-m-d H:i:s') / timeStamp()`

Fecha y hora actual con formato, o como timestamp Unix.

`arrayInString(array $array, string $string, string $delim = ' ')`

Indica si alguno de los elementos del arreglo aparece como palabra en el texto.

`getStringBetween(string $string, string $start, string $end, int $pos = 0)`

Retorna el texto que hay entre dos delimitadores.

`arrayToObject(array $array) / objectToArray($object)`

Convierten entre arreglo asociativo y objeto.

`isHttps(): bool`

Indica si la solicitud llegó por HTTPS, también cuando un proxy termina el TLS (cabecera `X-Forwarded-Proto`).

Responder a la vista

Respuesta a un APJCall()

```
// Vista: APJCall('solicitaSeccion', $('#valor').val(), 'B')
public function solicitaSeccion($valor, $seccion): void
{
    if ($fila = $this->modeloSeccion->find($valor)) {
        $this->jQ('#seccion')->val($fila[$seccion]);
    }
}
```

Respuesta a un APJSubmit()

```
// Vista: APJSubmit('formulario1', 'guardar')
public function guardar(): void
{
    $this->formToModel($this->modeloSeccion);
    if (!$this->modeloSeccion->basicValidation()) {
        $this->showWarnings($this->modeloSeccion->errors, 'Validación', $this->modeloSeccion-
>alias);
        return;
    }
    if ($this->modeloSeccion->guardar() === false) {
        $this->jError($this->modeloSeccion->errormsg);
        return;
    }
    $this->jInfo('Datos guardados');
}
```

> **Nota:** Use `jWarning()`/`showWarnings()` para los errores de validación (el usuario puede corregirlos) y `jError()`/`showErrors()` solo para errores reales. Y como `update()` o `upsert()` pueden retornar 0 filas sin que haya error, compare con `=== false`.

Instrucciones jQuery: jQ()

`$this->jQ(selector)` permite manipular la página desde PHP con la misma sintaxis de jQuery: se cambia `$(...)` por `$this->jQ(...)` y el punto por `->`. Las instrucciones se acumulan y se ejecutan en la vista, en el mismo orden, al recibir la respuesta.

```
$this->jQ('#identificador')->val('1');           // $('#identificador').val('1')
$this->jQ('#div2')->html($tabla)->show();
$this->jQ('.tabla')->hide();
$this->jQ('#formul')->resetForm();
$this->jQ('#guardar')->prop('disabled', true);
```

Se puede usar cualquier método de jQuery (https://api.jquery.com/) y los de formulario de jqajaj: `resetForm()`, `clearForm()` y `clearFields()`.

Diálogos desde el controlador

Los mismos diálogos de la vista, con los mismos argumentos. El callback se indica como texto (el nombre de una función global de la vista):

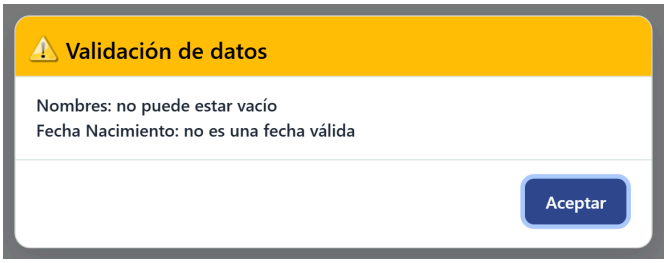
```
$this->jInfo('Registro guardado');
$this->jWarning('El RUT no es válido', 'Validación');
$this->jError('No se pudo eliminar el registro');
$this->jConfirm('¿Eliminar este registro?', 'Confirme', 'APJCall', ['eliminar', $id]);
$this->jPrompt('Motivo de la baja', 'Justifique', 'APJCall');
$this->jProcess('Procesando...', 'Espere', 'blink');
$this->jClose();
```

Arreglos de mensajes

`showWarnings(array $warnings, ?string $title = '', ?array $alias = [])`

Muestra todos los mensajes en un solo `jWarning()`, uno por línea, con el formato “campo: mensaje”. Con `$alias` usa el nombre descriptivo de cada campo (por ejemplo `$modelo->alias`). Es el que corresponde a

`$modelo->errors` después de `basicValidation()`.



`showErrors(array $errors, ?string $title = '', ?array $alias = [])`

Igual, con formato de `jError()`.

`showMessages(array $messages, string $title, string $type)`

Muestra una lista de mensajes; `$type` es `'Info'`, `'Warning'` o `'Error'`.

Otras respuestas

`APJSubmit(string $form, string $action, ...$args) / APJCall(string $method, ...$args)`

Hacen que la vista ejecute un `APJSubmit()`/`APJCall()` apenas recibe la respuesta. Útil, por ejemplo, para refrescar una grilla después de guardar, reenviando el formulario de filtros tal como lo tiene el usuario.

```
public function eliminar(int $id): void
{
    if ($this->modeloContactos->delete($id) !== false) {
        $this->APJSubmit('filtros', 'buscar');   // refresca la grilla con los filtros actuales
    }
}
```

`jScript(string $script)`

Ejecuta JavaScript en la vista. Prefiera `jQ()` y los métodos anteriores; use `jScript()` solo para lo que no se pueda expresar de otra forma.

`jShowDown(string $input, string $container, ?string $html = null)`

Pone el contenido `$html` en el elemento `$container` y lo muestra justo bajo el campo `$input` (ambos por id). Sirve para listas de búsqueda que se despliegan bajo el campo donde se escribe.

`jQ::setData($key, $value, ?string $callBack = null)`

Envía un dato a la vista (ver `APJ.data` en la API JavaScript).

APJHtmlGen

Genera HTML desde el controlador con una sintaxis encadenada, en lugar de concatenar textos. Declare la propiedad en el controlador antes de usarla:

```
private ?APJHtmlGen $gen = null;
// en el constructor:
$this->gen = new APJHtmlGen();
```

Método	Descripción
`create($tag)`	Reinicia el contenido y abre la etiqueta `$tag`.

Método	Descripción
`start()`	Reinicia el contenido, sin abrir etiquetas.
`add($tag, $closeLast = true)`	Abre una etiqueta (`br` y `hr` se cierran solas).
`attr($attr, $value)`	Agrega cualquier atributo a la etiqueta abierta.
`id()`, `clas()`, `name()`, `value()`, `title()`, `type()`, `style()`, `src()`	Atajos para esos atributos (`clas()` asigna `class`).
`onclick()`, `onchange()`, `onmouseover()`	Atajos para esos eventos.
`text($value)`	Agrega texto dentro de la etiqueta.
`literal($value)`	Agrega texto tal cual, sin cerrar la etiqueta abierta.
`preClose()`	Cierra la apertura de la etiqueta actual (`>`).
`close()` / `closeAll()`	Cierra la última etiqueta abierta / todas.
`end($closeAll = true)`	Cierra todo y retorna el HTML.
`getContent()`	Retorna el HTML generado hasta ahora, sin cerrar nada.

```
public function grillaContactos(string $pais): string
{
    $this->gen->start();
    foreach ($this->modeloContactos->porPais($pais) ?: [] as $row) {
        $rowf = $this->modeloContactos->setFormat($row);
        $this->gen->add('tr')->onclick("APJCall('editaContacto', {$row['id']})");
        foreach (['nombres', 'fecha_nacimiento', 'activo'] as $campo) {
            $this->gen->add('td')->text(htmlspecialchars((string) $rowf[$campo]))->close();
        }
        $this->gen->close();
    }
    return $this->gen->end();
}
```

Importante: `text()`, `value()` y `attr()` no escapan el contenido. Si el dato viene del usuario o de la base de datos, páselo por `htmlspecialchars()`.

El modelo (APJModel)

El modelo se encarga del acceso a la base de datos, las validaciones y la lógica de negocio. Hereda de `APJPDO`, que maneja la base de datos con PDO y enlaza los parámetros de las consultas (data binding), lo que evita la inyección de SQL. Además ofrece un ORM básico para las operaciones habituales.

Lo normal es un modelo por tabla, pero un modelo también puede cambiar de tabla, no tener ninguna u operar sobre varias con SQL.

Modelo en desarrollo

Archivo `Models/Contactos.php`:

```
<?php
namespace Models;

use APJ\APJModel;

class Contactos extends APJModel
{
    public function __construct()
    {
        parent::__construct();
        $this->setTable('contactos');
        $this->toUpper = ['nombres', 'apellido_paterno'];
    }
}
```

`setTable()` lee la estructura de la tabla (`SHOW FULL COLUMNS`) cada vez que se instancia el modelo. Es cómodo mientras la tabla cambia, pero agrega una consulta a cada solicitud.

En el controlador se usa así:

```
use Models\Contactos;
...
$this->modeloContactos = new Contactos();
```

Modelo en producción

En producción la estructura se escribe en el propio modelo, en un método privado `defineStructure()`, y así no se consulta en cada solicitud:

```
class Contactos extends APJModel
{
    public function __construct()
    {
        parent::__construct();
        $this->defineStructure();
        $this->toUpper = ['nombres', 'apellido_paterno'];
    }

    private function defineStructure()
    {
        $this->table = "contactos";
        $this->structure = array('id' => array('Type' => 'int', 'Size' => 11, ...), ...);
        $this->alias = array('id' => 'Id', 'nombres' => 'Nombres', ...);
        $this->pk = array('id');
        $this->fullPk = array('`contactos`.id');
        $this->fields = array_keys($this->structure);
    }
}
```

El contenido de `defineStructure()` no se escribe a mano: lo genera `getStructureCode()`, o directamente la herramienta `apjtool.php` (ver más adelante). Si la tabla cambia, hay que volver a generarlo.

Propiedades

Propiedad	Descripción
`table`	Nombre de la tabla.
`structure`	Estructura de cada columna: `Type`, `Size`, `Decimals`, `Null`, `Key`, `Default`, `Extra`, `Comment`.
`fields`	Nombres de las columnas.
`pk`, `fullPk`	Columnas de la clave primaria, sin y con el nombre de la tabla.
`alias`	Nombre descriptivo de cada columna. Sale del comentario (`COMMENT`) de la columna en la base de datos, o se asigna con `setAlias()`.
`variables`	Valores actuales de las columnas. Se leen y asignan como propiedades: `$this->nombres`.
`errors`	Errores de validación, con el formato [columna => mensaje]. Se puede agregar errores propios.
`toUpper`, `toLower`	Columnas que se convierten a mayúsculas o minúsculas al asignarlas.
`trim`	Si se quitan los espacios al inicio y al final al asignar valores (por defecto `false`).
`error`, `errormsg`, `errornum`	Si la última operación falló, su mensaje (el de la base de datos) y su código.
`affected`, `numrows`	Filas afectadas por la última acción y filas retornadas por la última consulta.
`trans`	Si hay una transacción iniciada.
`lastPage`, `currentPage`, `previousPage`, `nextPage`	Resultado de la última paginación hecha con `paging()`.

Al asignar una columna (`$this->fecha = '2026-10-22'`) el valor se convierte según el tipo de la columna (número, fecha, etc.) y un texto vacío se guarda como `NULL`. Asignar algo que no es una columna no tiene

efecto (queda anotado en el log de PHP) y leer una columna sin valor retorna `null`.

Métodos de estructura y documentación

`setTable(string $table)`

Asocia el modelo a una tabla y lee su estructura de la base de datos.

`setAlias(array $names)`

Asigna los nombres descriptivos de las columnas, en el mismo orden que `fields`.

`getStructureCode(): string`

Retorna el código PHP para el cuerpo de `defineStructure()`, listo para pegar.

`getDocComment(bool $short = false): string`

Retorna un comentario con la tabla de la estructura, para documentar el modelo sobre la declaración de la clase.

`getCreateTableCode(): string`

Retorna el `CREATE TABLE IF NOT EXISTS` de la tabla.

`showStructure() / showModel(bool $short = false) / showCreateTable()`

Muestran lo mismo que los anteriores en un cuadro de texto en pantalla y detienen la ejecución. Solo para diagnóstico.

Validación

`basicValidation(): bool`

Valida los valores según la estructura: columnas obligatorias vacías, largo máximo, números, fechas y horas. Retorna **`true` si todo es válido** y `false` si hay errores, que quedan en `errors`.

Si una columna `NOT NULL` que no es clave primaria viene vacía y tiene un valor por defecto en la base de datos, `basicValidation()` le asigna ese valor por defecto.

> **Importante:** Esto incluye defaults que son expresiones, como `current_timestamp()` en una columna `timestamp`: se asignaría el texto literal y se guardaría una fecha inválida. Asigne usted esas columnas (por ejemplo con `currentDateTime()`) o quítelas de los valores antes de guardar.

La validación propia de cada tabla va en el modelo, sobrescribiendo o complementando `basicValidation()` en su método de guardado:

```
public function guardar(): int|bool
{
    if (!$this->basicValidation()) {
        return false;                       // los errores quedan en $this->errors
    }
    if ($this->fecha_termino < $this->fecha_inicio) {
        $this->errors['fecha_termino'] = 'no puede ser anterior al inicio';
        return false;
    }
    if ($this->id) {                        // con id: actualizar
        return $this->update();
    }
    return $this->insert() === false ? false : $this->lastId();
}
```

Para revisar duplicados use otra instancia del modelo (`new Contactos()`), no `$this->find()`: `find()` reemplaza los valores que está por guardar.

Otros métodos

`setFormat(array $row): array`

Retorna la fila con cada columna formateada según su tipo y `FORMATS` (fechas, números, booleanos). Las columnas que no son del modelo (por ejemplo de un JOIN) quedan tal cual.

`clearValues()`

Limpia los valores de las columnas.

`server_info() / server_version() / getDataBaseName()`

Información del servidor, versión de la base de datos y nombre de la base.

`setCharset(string $charset = 'utf-8')`

Charset para las conversiones de `toUpper`/`toLower`.

Métodos ORM

La condición dinámica $where

`find()`, `update()`, `delete()`, `select()` y `where()` aceptan la condición de varias formas:

Forma	Ejemplo	Equivale a
Número	`$this->find(15)`	`WHERE clave_primaria = 15`
Arreglo	`$this->delete(['pais' => 'CL', 'activo' => 0])`	`WHERE pais = :pais0 AND activo = :activo1` (con parámetros enlazados)
Texto	`$this->select("pais = 'CL' AND edad > 18")`	La condición tal cual
Nada	`$this->update()`	La clave primaria con el valor actual del modelo

> **Importante:** La forma de texto se usa tal cual: no incluya datos del usuario sin validarlos. Prefiera la forma de arreglo, que enlaza los valores como parámetros.

Escritura

`insert(bool $ignore = false, ?array $values = null): int|bool`

Inserta una fila con los valores del modelo o con el arreglo `$values`. Con `$ignore = true` usa `INSERT IGNORE`. Retorna las filas insertadas o `false`.

`update($where = '', $values = ''): int|bool`

Actualiza según la condición (por defecto, la clave primaria). Retorna las filas afectadas o `false`.

`upsert(?array $values = null): int|bool`

Inserta, o actualiza si ya existe la clave primaria o un índice único (`INSERT ... ON DUPLICATE KEY UPDATE`). A diferencia de `replace()`, nunca borra la fila existente.

`replace($values = ''): int|bool`

Como `insert()`, pero si ya existe la clave primaria o un índice único, borra la fila y la vuelve a insertar (`REPLACE INTO`). Prefiera `upsert()`.

`delete($where = '')`

Elimina según la condición (por defecto, la clave primaria).

`lastId(): int|false`

Último id autoincremental insertado. Pídalo justo después de `insert()`.

`truncate(?string $table = null) / drop(bool $temporary = false, ?string $table = null)`

Vacían o eliminan la tabla del modelo (o la indicada).

> **Importante:** `update()` y `upsert()` pueden retornar **0** sin que haya error: no había cambios que aplicar. Compruebe siempre con `=== false` (o con `$modelo->error`), nunca con `if (!$resultado)`.

```
if ($this->modeloContactos->update() === false) {
    $this->jError($this->modeloContactos->errormsg);
    return;
}
```

Lectura

`find($where = '')`

Busca una fila; la retorna como arreglo asociativo y además deja sus valores en el modelo (`$this->nombres`). Retorna `false` si no la encuentra.

```
// Modelo
public function estaActivo(int $id): bool
{
    $this->find($id);
    return (bool) $this->activo;
}
```

`all(?string $order = ''): array`

Todas las filas de la tabla, con orden opcional.

`select($condition = '', ?string $order = '')`

Las filas que cumplen la condición dinámica, con orden opcional.

`like(string $field, string $search): array`

Las filas cuya columna cumple un `LIKE` (incluya usted los `%`).

```
return $this->like('nombres', $texto . '%');
```

`count / sum / avg / min / max (string $field, $condition = null)`

Cuenta, suma, promedio, mínimo y máximo de una columna, con la condición dinámica opcional. `count()` retorna un entero.

`paging(string $query, int $limit = 20, int $page = 1)`

Retorna una página de la consulta (o de la tabla, si `$query` va vacío) y actualiza las propiedades de paginación.

Modificadores de consulta

`where()`, `order()`, `limit()` y `join()` preparan la siguiente consulta de `find()`, `all()` o `select()`:

```
$this->join('grupos', 'id_grupo', '=', 'id', 'LEFT');
$this->order('nombres');
$this->limit(50);
$filas = $this->select(['activo' => 1]);
```

Método	Descripción
`where($condition)`	Fija la condición para la siguiente consulta.
`order(string $order)`	Orden (`ORDER BY`).
`limit($limit)`	Máximo de filas (`LIMIT`); `0` = sin límite.
`join($table, $column, $comparator, $foreignKey, $type = 'INNER')`	Agrega un JOIN (`INNER`, `LEFT`, `RIGHT`, `LEFT OUTER`, `RIGHT OUTER`). Se pueden encadenar varios.
`clearJoins()`	Quita todos los JOIN.

> **Importante:** `order()`, `limit()` y `join()` **se mantienen** para las consultas siguientes del mismo modelo, hasta que se cambien (`order('')`, `limit(0)`, `clearJoins()`). La condición de `where()` sí se limpia en cada consulta.

SQL directo

`row(string $query, $params = [], int $fetchMode = PDO::FETCH_ASSOC)`

Retorna una fila de una consulta `SELECT`, `SHOW` o `DESCRIBE`, con parámetros enlazados. Los parámetros se indican **sin** dos puntos: `['pais' => $pais]`.

```
public function buscarDni(string $pais, string $dni): array|false
{
    $sql = "SELECT nombres, apellido_paterno FROM contactos WHERE pais = :pais AND dni = :dni";
    return $this->row($sql, ['pais' => $pais, 'dni' => $dni]);
}
```

`rows(string $query, $params = [], int $fetchMode = PDO::FETCH_ASSOC)`

Igual que `row()`, pero retorna todas las filas.

`nextRow(int $fetchmode = PDO::FETCH_ASSOC)`

La siguiente fila de la última consulta.

`execute(string $query, $params = null): int|false`

Ejecuta un `INSERT`, `UPDATE`, `REPLACE`, `DELETE` o `TRUNCATE` con parámetros enlazados; retorna las filas afectadas.

`query(string $query): int|false`

Ejecuta una consulta sin parámetros; retorna las filas afectadas o retornadas.

`bind() / bindArray() / clearBinding()`

Enlazan parámetros manualmente, uno a uno o con un arreglo, y limpian los enlaces.

Los modos de lectura (`$fetchMode`) son los de PDO: `FETCH_ASSOC` (por defecto), `FETCH_NUM`, `FETCH_BOTH`, `FETCH_OBJ` y `FETCH_CLASS`.

Transacciones

Una transacción agrupa varias operaciones para que se apliquen todas o ninguna. Requiere tablas InnoDB.

Método	Descripción
`beginTrans()`	Inicia la transacción.
`commitTrans()`	Confirma la transacción.
`rollBack()`	Deshace la transacción.
`endTrans()`	Intenta confirmar y, si falla, deshace.

Todos los modelos comparten una misma conexión, así que una transacción iniciada en un modelo incluye lo que hagan los demás modelos hasta confirmarla o deshacerla:

```
public function guardarConDetalle(array $detalle): bool
{
    if (!$this->basicValidation()) {
        return false;
    }
    $this->beginTrans();
    if ($this->insert() === false) {
        $this->rollBack();
        return false;
    }
    $idMaestro = $this->lastId();
    $modeloDetalle = new Detalle();
    foreach ($detalle as $fila) {
        $fila['id_maestro'] = $idMaestro;
        if ($modeloDetalle->insert(false, $fila) === false) {
            $this->errormsg = $modeloDetalle->errormsg;
            $this->rollBack();
            return false;
        }
    }
    return $this->commitTrans();
}
```

> **Nota:** No anide transacciones: si un método que ya usa `beginTrans()` llama a otro que también la inicia, el segundo `beginTrans()` falla. Controle la transacción en un solo nivel.

Herramienta de modelos (apjtool.php)

`apjtool.php` (con `APJ/APJModelTool.php`) muestra todos los modelos de la carpeta `Models` y automatiza su paso de desarrollo a producción. Solo funciona con `DEVELOPMENT = true` y pide sesión iniciada. Ábrala en http://localhost/<proyecto>/apjtool.php.

Estado	Significado
DEV	El modelo usa `setTable('tabla')`.
PROD	El modelo usa `defineStructure()`.
DEV + VIEW	`setTable()` apunta a una vista de la base de datos: solo se puede documentar.
DEV + MISSING	`setTable()` apunta a una tabla que no existe en la base: no hay acciones.
?	No se reconoce el patrón, por ejemplo `setTable($variable)`.

Acción	Qué hace
Promote	Reemplaza `setTable()` por `defineStructure()` y genera ese método con la estructura actual.
+ Doc	Además agrega (o actualiza) el comentario con la tabla de la estructura sobre la clase.
+ Doc + Create	Además agrega un método `createTableIfNotExists()` con el `CREATE TABLE`.
Doc Only	Solo agrega o actualiza el comentario de documentación; el modelo sigue en DEV.
Demote	Vuelve el modelo a DEV: restaura `setTable()` y quita `defineStructure()`, `createTableIfNotExists()` y el comentario generado.

También hay acciones para promover, documentar o revertir todos los modelos a la vez. La herramienta modifica los archivos de `Models/`; respalde antes si no usa control de versiones.

Control de sesión (APJSession)

Clase estática que controla la sesión de los usuarios. El nombre de la sesión se deriva de `APPNAME` (cifrado con md5), así que varias aplicaciones APJ pueden convivir en el mismo dominio. La cookie de sesión se crea con `HttpOnly`, `SameSite=Strict` y `Secure` cuando la conexión es HTTPS.

Iniciar sesión (login)

Su controlador de login, después de validar al usuario, debe iniciar la sesión y asignar dos valores:

```
APJSession::start(APPNAME);
$_SESSION['id']  = $usuario['id'];
$_SESSION['app'] = array_map('md5', $controladoresPermitidos);   // ['consulta', 'contactos',
...]
```

Valor	Contenido
`$_SESSION['id']`	Id único del usuario. Lo asigna usted.
`$_SESSION['app']`	Controladores a los que tiene acceso, como el **md5 del nombre del controlador** (el nombre del archivo sin `.php`). Lo asigna usted.
`$_SESSION['userAgent']`, `$_SESSION['IPaddress']`	Navegador e IP (cifrados) con que se inició la sesión. Los asigna `APJSession`.
`$_SESSION['last_activity']`	Hora de la última actividad, para el límite de inactividad.

Luego, en cada controlador, `sessionControl()` verifica que la sesión siga activa, que el navegador sea el mismo y, salvo con `sessionControl(false)`, que `md5(getController())` esté en `$_SESSION['app']`. Si algo falla, redirige a `LOGIN`. La sesión caduca después de `SESSION_LIMIT` segundos sin actividad. Un cambio de IP no cierra la sesión (las redes móviles cambian de IP con frecuencia); un cambio de navegador sí.

Métodos

Método	Descripción
`active(string $name): bool`	Indica si la sesión sigue activa y es del mismo navegador; renueva la actividad.
`start(string $name, int $limit = 0, ?string $path = null, ?string $domain = null, ?bool $secure = null)`	Inicia o retoma la sesión. `$limit = 0` usa `SESSION_LIMIT`.
`destroy()`	Cierra la sesión y borra su cookie (cierre de sesión del usuario).
`destroySession()` / `deleteSessionCookie()`	Las dos partes de `destroy()` por separado.
`setLoginCookie($sistema, $token, $dias = 30)`	Crea una cookie “recordarme” con un token.
`verifyLoginCookie($sistema): ?string`	Retorna el token de la cookie si es válido.
`deleteRememberMeCookies($sistema)`	Borra las cookies “recordarme”.

Método	Descripción
`setSessionTokenModel($modelo, $metodo)` / `setSessionPDO($pdo)`	Definen cómo se valida el token: con un método de un modelo suyo o con una tabla `session_tokens`.
`validateToken($token): bool`	Valida un token con lo definido arriba.

Registro de errores (APJLog)

Los errores de base de datos y las excepciones no capturadas en los métodos del controlador se registran en `APJ/Logs/`, en un archivo por día (`AAAA-MM-DD.txt`), con la hora, la clase y método de origen, el mensaje y, si corresponde, el SQL. También puede usarlo en su código:

```
$log = new \APJ\APJLog();
$log->errorLog('No se pudo enviar el correo a ' . $email);
```

> **Nota:** Proteja la carpeta `APJ/Logs/` para que no sea accesible desde el navegador, porque puede contener SQL y datos.

Buenas prácticas

- **Vista fija, controlador dinámico:** la vista tiene la estructura; el controlador regenera solo el fragmento que cambia (por ejemplo el `<tbody>`) con `APJHtmlGen` y lo inserta con `$this->jQ('#id')->html(...)`.
- **La lógica en PHP:** nada de JavaScript propio en la vista para mostrar, ocultar o limpiar; exprese todo con `$this->jQ(...)`. Para disparar un `APJSubmit()`/`APJCall()` desde el controlador use `$this->APJSubmit()`/`$this->APJCall()`, no `jScript()`.
- **Validación en el modelo:** con `basicValidation()` (true = válido) y sus reglas propias en el método de guardado.
- **Después de modificar datos**, refresque la grilla reenviando el formulario de filtros (`$this->APJSubmit('filtros', 'buscar')`) en vez de recalcularla con valores por defecto.
- `=== false` para detectar errores de `update()`, `upsert()`, `insert()` y `execute()`.
- **`APJCall()` no envía el formulario**; si necesita sus datos, use `APJSubmit()` o pase los valores como argumentos.
- **`APJSubmitSilent()`/`APJCallSilent()`** en los `onchange`/`onblur` que solo consultan; las versiones normales para guardar.
- **Declare las propiedades** del controlador y del modelo (PHP 8.2 o superior).
- **Modelos en producción** con `defineStructure()` (use `apjtool.php`) y `DEVELOPMENT = false`.
- **Escape los datos** que pone en HTML (`htmlspecialchars()`).
- **Versione los archivos de jqajaj** en las vistas (`?ver=2.0.7`) y suba siempre juntos el JS y el CSS.

Historial de versiones

Versión	Cambios principales
2.1 (oct-2026)	Preparada para PHP 8.5 (ver “Novedades”). jqajaj 2.0.7 y `jqajaj.css` autosuficiente. Versión multilenguaje al día.
jqajaj 2.0.5–2.0.7 (oct-2026)	`APJSubmitSilent()`/`APJCallSilent()`; diálogos con estructura propia; teclado confinado al diálogo; corrección de foco con ventanas modales de la página.
2.0 (ago–oct 2026)	`upsert()`; `getStructureCode()`, `getDocComment()`, `getCreateTableCode()`; `apjtool.php`; `$this->APJSubmit()`/`$this->APJCall()`; `absoluteUrl()`; `basicValidation()` retorna `true` si es válido; columnas generadas excluidas al guardar; la sesión ya no se invalida por cambio de IP; correcciones en `format()`, `getParameters()`, `execute()` y en los argumentos de `APJ:{...}`.
2.0 (dic-2024)	Primera versión 2: namespaces, PHP 8, `APJ.init()`, `jQAction` con `JsonSerializable`.